# PORTFOLIO TECHNICAL ARCHITECTURE & 3D GLOBE ENGINE

Comprehensive Code Breakdown, System Architecture Analysis, and Interactive WebGL Implementation Defense Guide


## Presenter: Leroy Matienga

**Academic Standing:** Undergraduate Computer Science Student

**Specialization:** Software Engineering, Backend Systems & 3D Web Graphics




**Portfolio Codebase:** Full-Stack Web Application (HTML5, CSS3, JS, Three.js, Flask, SQLite)

**Presentation Objective:** Defend the design decisions, code organization, full-stack REST API, SQLite schema, security mitigations, and the mathematical implementation of the real-time 3D Earth Globe.

**Date of Defense: October 2026**

| System Dimension | Implementation Technology | Core Architectural Function |
|---|---|---|
| **Frontend Presentation** | HTML5, CSS3, ES6 JavaScript | Responsive glassmorphism UI, sliding magnetic pill navigation, dynamic scroll-spy, animated skill meters. |
| **3D Interactive Globe** | Three.js, WebGL, Custom GLSL | Real-time day/night simulation based on UTC Sun vector, procedural starfield, great-circle flight hyper-dive. |
| **Backend Web Server** | Python 3, Flask 3.1, CORS | REST API routing, input sanitization against XSS, directory traversal guards, CV distribution. |
| **Data Persistence** | SQLite 3 (Built-in engine) | Atomic transactional logging, parameterized query execution (SQLi immunity), sliding-window rate tracking. |
| **Automated Messaging** | Python smtplib, MIME Multipart | Automated branded HTML contact notifications with non-blocking graceful degradation. |
| **Configuration & Security** | python-dotenv, .env isolation | 12-Factor App secrets isolation, IP-based sliding-window rate limiter preventing bot spam. |

**[KEY PRESENTATION TAKEAWAY]**

This presentation guide is structured into self-contained modules. You can navigate directly to **Pages 7-9** for the complete, step-by-step mathematical and shader breakdown of the **3D Interactive Earth Globe**, or to **Pages 11-13** for word-for-word presentation speaking scripts, anticipated judge defense Q&A, and live demo checklists.

# 1. System Architecture & Engineering Principles

Leroy Matienga's portfolio is engineered as a high-performance full-stack client-server application. It couples an immersive 3D WebGL computer graphics interface on the client with a secure, rate-limited, and persistent Python backend. The architecture adheres to four foundational software engineering principles:

- **1. Modularity & Separation of Concerns:** Presentation markup (HTML), visual styling (CSS), client-side interactivity and 3D rendering (JavaScript/Three.js), API routing (Flask), database management (SQLite), and external communication (SMTP) are isolated into dedicated, decoupled modules.
- **2. Defensive Security Architecture:** All client inputs are treated as untrusted. The backend enforces input sanitization via html.escape() to eliminate Cross-Site Scripting (XSS), uses strict regex pattern checking, employs parameterized SQL queries to prevent SQL Injection (SQLi), guards against directory traversal attacks, and limits request frequencies through an IP-based sliding window.
- **3. Hardware-Accelerated 3D Web Graphics:** Rather than using pre-rendered video clips or 2D raster maps, the portfolio runs a real-time WebGL rendering pipeline via Three.js with custom GLSL shaders that calculate solar illumination vectors directly on the client's GPU at 60 FPS.
- **4. Graceful Degradation & Non-Blocking Resilience:** If external services (such as SMTP mail servers) encounter network dropouts or authentication timeouts, the core application handles the error gracefully, ensuring that database records remain intact and the user receives a reliable response.

## Full-Stack Component Interaction Pipeline




**Presenter Talking Script: Explaining System Architecture to Examiners**

*"Judges, our architecture follows strict separation of concerns. The frontend combines high-performance Three.js WebGL graphics with modern CSS glassmorphism, while the backend is an enterprise-patterned Flask microservice backed by SQLite. We specifically engineered defensive mechanisms at every boundary: client input is sanitized, requests are rate-limited per IP, SQL queries use strict parameterization, and external services degrade gracefully without crashing the server."*

# 2. Frontend Semantic & Visual Architecture

## 2.1 index.html -- Semantic Markup & DOM Hierarchy

**File Role:** Serves as the Single-Page Application (SPA) foundation. It establishes the semantic document structure, imports vector icon and 3D graphics libraries via CDNs, and exposes accessible target nodes for JavaScript manipulation.

- **Header Meta & CDN Imports (Lines 1-20):** Declares HTML5 doctype, UTF-8 encoding, and responsive viewport sizing. Imports *Font Awesome 6.5.0* for UI icons, *Google Poppins* typography (weights 300 to 700), and loads *Three.js (r128)* along with *OrbitControls.js* for 3D globe rendering.
- **Ambient Slideshow Background (Lines 23-29):** Encapsulates 3 background slides (slide-1, slide-2, slide-3) beneath a dark radial overlay (.bg-overlay) to produce a continuous background crossfade without impeding user interaction.
- **Navigation Bar (Lines 31-50):** Semantic <header> element containing Leroy's logo badge, mobile hamburger toggle button, and the floating navigation links with an empty  pill that slides dynamically.
- **Hero Section (Lines 52-84):** Centers Leroy's portrait (profile2.jpeg) within a dual-layer glowing container (avatar-glow and avatar-border). Embeds the dynamic typewriter target (#typewriter), bio statement, call-to-action buttons (Download CV linking to /api/download-cv, Contact link), and GitHub/LinkedIn social anchors.
- **About Section (Lines 86-112):** 3 glassmorphism cards detailing Education (Computer Science), Development (Software Engineering), and Specialization (Backend & AI Systems).
- **Interactive 3D Portfolio Globe Frame (Lines 114-186):** The dedicated 3D viewport wrapper (#globeWrapper). Houses the Top Control Bar (Orbit reset button, Real-Time Sun indicator with pulsing LED, Fullscreen toggle), the Atmospheric Entry Flight HUD (#globeFlightHud), the WebGL container (#globe-container), the Quick-Dive dock pills, and the status modal dialog (#statusModal).
- **Skills Section (Lines 188-227):** Four skill categories (Python OOP, SQL Database, Web Frontend, Git Version Control) featuring animated percentage meters (--w: 90%).
- **Contact Section & Footer (Lines 229-266):** Asynchronous contact form (#contactForm) with name, email, subject, and message inputs, plus an animated submission button with paper-plane icon.

## 2.2 style.css -- Design Tokens, Glassmorphism & HUD Styling

**File Role:** Controls the visual language, dark space theme, glassmorphic blur effects, hardware-accelerated animations, and responsive media queries across all viewport sizes.

- **CSS Root Variables (Lines 1-11):** Standardizes the color palette: --bg-primary: #0f1013 (deep space dark), --accent-color: #2563eb (royal blue), --accent-glow: rgba(37,99,235,0.45), --text-main: #f8fafc, and --transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1) for smooth animations.
- **Cyclic Background Slideshow (Lines 32-82):** Uses @keyframes slideFade over an 18-second cycle with staggered 6-second animation delays. The dark radial gradient overlay and backdrop-filter: blur(4px) ensure text readability remains high.
- **Glassmorphism Architecture (Lines 84-92):** The .glass-card utility class combines background: var(--glass-bg), backdrop-filter: blur(12px), and a 1px subtle white alpha border (rgba(255,255,255,0.12)) to give floating surfaces depth and elegance.
- **Sliding Magnetic Indicator Pill (Lines 130-175):** Positions the indicator pill inside the navigation capsule with position: absolute. Smooth motion is achieved using GPU transforms (translateX and width) driven by cubic bezier curves.
- **3D Space Frame & Top Bar (Lines 403-498):** Styles the 640px height 3D globe wrapper with a space radial background (#070913 to #020308), inset blue shadow, and positions the Top Bar (z-index: 35) with pointer events passed through to controls.
- **Atmospheric Flight HUD (Lines 501-553):** When a dive begins, this HUD translates down into the viewport with a glowing cyan border (#38bdf8), scanning laser gradient (@keyframes hudScan), and floating airplane icon (@keyframes hudPlane).
- **Responsive Breakpoints (Lines 960-1444):** Employs CSS media queries at 1024px, 768px, and 480px to collapse the navbar into a mobile slide-out drawer, stack the hero grid vertically, and scale down the globe height for smooth mobile performance.

# 2.3 script.js -- Client Logic & DOM Event Controllers

**File Role:** Serves as the primary frontend controller. Manages DOM events, sliding navigation tracking, typewriter effects, asynchronous contact submissions, modal state machines, and the Three.js 3D Earth Globe.

- **Module 1: Sliding Magnetic Capsule Indicator (Lines 1-66):** Tracks mouse movement and drag states across navigation items. Calculates horizontal position using targetRect.left - containerRect.left and adjusts indicator pill width to match the hovered or active tab.
- **Module 2: Dynamic ScrollSpy Engine (Lines 67-91):** Listens to window scroll events with an offset of +220px. Evaluates whether the current scroll position falls within the boundary [sectionTop, sectionTop + sectionHeight], auto-activating the matching nav link.
- **Module 3: Typewriter State Machine (Lines 93-123):** Cycles through role titles ("Software Developer!", "CS Student!", "Python Engineer!", "Tech Enthusiast!") using an alternating character indexing loop with variable delays for typing (100ms), deleting (50ms), and word pausing (1500ms).
- **Module 4: IntersectionObserver Skill Meters (Lines 138-150):** Observes skill progress bars with a 30% viewport visibility threshold (threshold: 0.3). When scrolled into view, it attaches the .animated class, triggering smooth CSS transitions from 0% to the target width.
- **Module 5: Asynchronous Contact Form AJAX Handler (Lines 152-210):** Intercepts form submit events, prevents full-page browser reloads via e.preventDefault(), transforms button state into a loading spinner, and issues a POST request to /api/contact with JSON payload. Dynamically renders success or error messages without refreshing.
- **Module 6: Global Status / Progress Modal Manager (Lines 218-316):** Controls modal visibility. Crucially, it dynamically reparents the modal into #globeWrapper so that when the user is in HTML5 Fullscreen mode, the modal remains accessible on the top-layer stack instead of being occluded.

## Contact Form AJAX Submission Workflow

```
// Asynchronous AJAX Request Workflow in script.js
const formData = {
    name: document.getElementById("name").value.trim(),
    email: document.getElementById("email").value.trim(),
    subject: document.getElementById("subject").value.trim(),
    message: document.getElementById("message").value.trim(),
};
const response = await fetch("/api/contact", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(formData),
});
const result = await response.json();
if (response.ok && result.success) {
    btn.innerHTML = `<span>Sent Successfully!</span> <i class="fa-solid fa-check"></i>`;
    btn.style.backgroundColor = "#10b981";
    form.reset();
} else {
    btn.innerHTML = `<span>${result.error || "Failed to send"}</span> <i class="fa-solid fa-xmark"></i>`;
    btn.style.backgroundColor = "#ef4444";
}
```

**[CODE QUALITY HIGHLIGHT: NON-BLOCKING EVENT LOOPS]**

Notice how script.js handles all heavy animations (the starfield drift, cloud rotation, raycasting, and camera flight curves) using requestAnimationFrame(animate) rather than setInterval. This synchronizes rendering with the monitor's vertical refresh rate, completely eliminating frame tearing and minimizing battery consumption on laptops and mobile devices.

# 2.4 app.py -- Flask Backend Server & Security Pipeline

**File Role:** The central backend server powered by Python 3 and Flask. It serves static assets, enforces rate limiting, sanitizes untrusted input, logs contact messages to SQLite, dispatches email notifications, and guards administrative endpoints.

| Route / Method | Authorization | Functionality & Security Controls |
|---|---|---|
| `GET /` | Public | Delivers index.html from the project root via send_from_directory(). |
| `GET /<path:filename>` | Public | Serves static assets (CSS, JS, images). Includes a **Directory Traversal Guard** checking if '..' in filename: abort(403). |
| `POST /api/contact` | Rate Limited | Processes contact submissions. Runs IP sliding-window rate limit check, input sanitization via html.escape(), regex email validation, records to SQLite, and triggers email. |
| `GET /api/download-cv` | Public | Safely checks for file existence before streaming the CV attachment to the user with standard Content-Disposition headers. |
| `GET /api/messages` | Admin API Key | Administrative endpoint. Validates request.headers.get('X-API-Key') == Config.ADMIN_API_KEY before returning logged messages as JSON. |

## Security Architecture & Input Validation Logic

- **Proxy-Aware IP Resolution:** Uses get_client_ip() to extract the true client IP from request.headers.get('X-Forwarded-For') when deployed behind reverse proxies (such as Nginx, Cloudflare, or AWS ALBs), falling back to request.remote_addr.
- **Cross-Site Scripting (XSS) Sanitization:** All textual inputs are passed through html.escape(text.strip()). Malicious HTML injection payloads (e.g., <script>alert(1)</script>) are escaped into harmless HTML entities (<script>).
- **Input Length & Regex Boundaries:** Verifies email RFC formatting using standard regex patterns and enforces strict length boundaries (Name: 2-100 chars, Subject: 2-200 chars, Message: 10-5000 chars) to prevent buffer overflows or denial-of-service via massive payloads.

```
# Critical Security & Validation Snippet from app.py
@app.route("/api/contact", methods=["POST"])
def handle_contact():
    client_ip = get_client_ip()
    # Rate Limiting Check (5 requests per 60 seconds per IP)
    if count_recent_messages(client_ip, Config.RATE_LIMIT_WINDOW_SECONDS) >= Config.RATE_LIMIT_MAX_REQUESTS:
        return jsonify({"success": False, "error": "Too many messages. Please try again later."}), 429
    data = request.get_json() or {}
    name = sanitize_input(data.get("name", ""))
    email = sanitize_input(data.get("email", ""))
    subject = sanitize_input(data.get("subject", ""))
    message = sanitize_input(data.get("message", ""))
    if not is_valid_email(email) or len(message) < 10:
        return jsonify({"success": False, "error": "Invalid form fields."}), 400
    saved = save_message(name, email, subject, message, ip_address=client_ip)
    email_sent = send_notification(name, email, subject, message)
    return jsonify({"success": True, "message": "Sent successfully!"}), 201
```

# 2.5 Database Persistence & Messaging Services

## 2.5.1 config.py -- Centralized Configuration Management

**File Role:** Implements the Twelve-Factor App methodology for configuration management. It uses python-dotenv to read environment variables from an uncommitted .env file into a clean Python Config object.

```
class Config:
    SECRET_KEY = os.getenv("SECRET_KEY", "change-this-to-a-random-secret-key")
    DEBUG = os.getenv("FLASK_DEBUG", "False").lower() in ("true", "1", "yes")
    DATABASE_PATH = os.getenv("DATABASE_PATH", "portfolio.db")
    SMTP_SERVER = os.getenv("SMTP_SERVER", "smtp.gmail.com")
    SMTP_PORT = int(os.getenv("SMTP_PORT", "587"))
    SMTP_EMAIL = os.getenv("SMTP_EMAIL", "")
    SMTP_PASSWORD = os.getenv("SMTP_PASSWORD", "")
    NOTIFICATION_EMAIL = os.getenv("NOTIFICATION_EMAIL", "")
    ADMIN_API_KEY = os.getenv("ADMIN_API_KEY", "change-this-to-a-secure-key")
    CV_FILENAME = os.getenv("CV_FILENAME", "Leroy_Matienga_CV.pdf")
    RATE_LIMIT_MAX_REQUESTS = int(os.getenv("RATE_LIMIT_MAX_REQUESTS", "5"))
    RATE_LIMIT_WINDOW_SECONDS = int(os.getenv("RATE_LIMIT_WINDOW_SECONDS", "60"))
```

## 2.5.2 models.py -- SQLite Database Layer & SQLi Immunity

**File Role:** Encapsulates all relational database operations using Python's native sqlite3 library. It manages database connection factories, table schema migrations, and transactional inserts.

- **Connection Factory (get_db_connection()):** Opens connections to portfolio.db and attaches conn.row_factory = sqlite3.Row. This enables column access via dictionary keys (e.g., row['email']) as well as numerical indices.
- **Idempotent Schema Initialization (init_db()):** Executes CREATE TABLE IF NOT EXISTS contacts on server startup. The table schema includes: id (INTEGER PK AUTOINCREMENT), name (TEXT), email (TEXT), subject (TEXT), message (TEXT), ip_address (TEXT), is_read (INTEGER DEFAULT 0), and created_at (TIMESTAMP).
- **SQL Injection Immunity (save_message()):** All values are passed into SQL statements as parameterized placeholders (?). The database engine treats all parameters purely as literal values, rendering SQL injection exploits impossible.
- **Sliding-Window Rate Query (count_recent_messages()):** Queries message frequency using SQLite's native date functions: WHERE ip_address = ? AND created_at >= datetime('now', '-60 seconds'). This provides lightweight rate limiting without external Redis dependencies.

## 2.5.3 email_service.py -- Automated SMTP Notification Engine

**File Role:** Dispatches branded email alerts to Leroy when visitors submit contact inquiries. Built using Python's native smtplib and email.mime packages.

- **Dual-Part MIME Construction:** Generates a MIMEMultipart('alternative') container holding both a plain-text fallback and a rich HTML layout.
- **TLS Encryption:** Connects over port 587 and executes server.starttls() to ensure end-to-end transport encryption before transmitting credentials and message bodies.
- **Non-Blocking Fail-Safe Execution:** Wrapped completely inside nested try-except blocks. If SMTP credentials are empty or the mail server experiences latency, it logs to stderr but returns False without failing the client's HTTP request.

# 3. The Masterpiece: 3D Real-Time Earth Globe Engine (Part 1)

The focal point of Leroy's portfolio is the **Interactive 3D Real-Time Day & Night Earth Globe**. Unlike standard portfolios that show static images or 2D Google Maps, this system runs a hardware-accelerated WebGL rendering engine inside Three.js. It features astronomical solar position calculations, custom GLSL shaders, dynamic canvas billboards, and an orbital great-circle camera flight engine.

**[PRESENTER ANCHOR CONCEPT]**

When explaining the 3D globe to the audience tomorrow, emphasize that the Earth is not a flat textured sphere with a static light. **It computes the actual mathematical position of the Sun right now in real time using the current UTC hour and day of the year**, blending NASA Blue Marble daylight with golden night city lights via a custom GLSL GPU fragment shader.

## Step 1: Foundational 3D Scene Architecture

The 3D environment is initialized in initEarthGlobe() inside script.js using four foundational Three.js constructs:

- **1. THREE.Scene:** The 3D world container. Configured with a deep space black background color: scene.background = new THREE.Color(0x020308).
- **2. THREE.PerspectiveCamera:** Configured with a 45 deg Field of View (FOV), aspect ratio matching the container dimensions, a near clipping plane of 0.1, and far plane of 2000. Positioned initially at orbital coordinates (x: 0, y: 4, z: 24).
- **3. THREE.WebGLRenderer:** Configured with { antialias: true, alpha: false }. Pixel ratio is capped at Math.min(window.devicePixelRatio, 2) to maintain smooth 60 FPS performance on high-DPI Retina screens without GPU overheating.
- **4. THREE.OrbitControls:** Enables smooth mouse drag rotation, pinch zoom, and inertia damping (dampingFactor: 0.05). Distance is clamped between minDistance: 7.5 (preventing surface clipping) and maxDistance: 45 (preventing lost camera orbit). Auto-rotation is set to 0.65 rad/s.

## Step 2: Celestial Mechanics & Real-Time Solar Calculations

To achieve accurate real-time day and night illumination, the application calculates the **subsolar point** (the exact geographic latitude and longitude where the Sun is directly overhead) using live UTC time:

```
// 1. Calculate Day of Year and UTC Decimal Hours
const now = new Date();
const utcHours = now.getUTCHours() + now.getUTCMinutes() / 60 + now.getUTCSeconds() / 3600;
const startOfYear = new Date(Date.UTC(now.getUTCFullYear(), 0, 1));
const dayOfYear = Math.floor((now - startOfYear) / 86400000);
// 2. Solar Declination Angle (Earth's 23.44 deg axial tilt variation over 365.25 days)
const declination = -23.44 * Math.cos(((2 * Math.PI) / 365.25) * (dayOfYear + 10));
// 3. Subsolar Longitude (Sun crosses the Prime Meridian 0 deg at exactly 12:00 UTC)
const subsolarLon = -(utcHours - 12) * 15;
```

**Mathematical Interpretation:** Earth rotates 360 deg in 24 hours, which equates to **15 deg of longitude per hour**. At 12:00 UTC, the Sun is at longitude 0 deg. At 18:00 UTC, the Sun has moved westward by (18 - 12) * 15 deg = 90 deg W (over the Americas). The calculated solar coordinates (lat, lon) are converted into a 3D unit vector and transformed by Earth's current rotational Euler angle to ensure the sunlight dynamically matches real-world planetary orientation.

# 3. The Masterpiece: 3D Real-Time Earth Globe Engine (Part 2)

## Step 3: Custom GLSL Day/Night Shader Material

Standard Three.js materials (such as MeshStandardMaterial) only support a single diffuse texture. To achieve seamless day-to-night transitions with glowing city lights and atmospheric scattering, Leroy implemented a custom **GLSL ShaderMaterial** with custom vertex and fragment programs:

```
// Vertex Shader: Transforms normals into world coordinates
varying vec2 vUv;
varying vec3 vNormal;
varying vec3 vWorldNormal;
void main() {
    vUv = uv;
    vNormal = normalize(normalMatrix * normal);
    vWorldNormal = normalize((modelMatrix * vec4(normal, 0.0)).xyz);
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
}
// Fragment Shader: Blends Day, Night, Twilight Amber Glow, and Fresnel Rim
uniform sampler2D uDayTexture;     // NASA Blue Marble Earth Texture
uniform sampler2D uNightTexture;   // NASA Earth Night City Lights Texture
uniform vec3 uSunDirection;        // Live calculated Sun Direction Vector
void main() {
    vec3 dayColor = texture2D(uDayTexture, vUv).rgb;
    vec3 nightColor = texture2D(uNightTexture, vUv).rgb;
    // 1. Dot Product: Cosine of angle between surface normal and sun direction
    float sunDot = dot(vWorldNormal, normalize(uSunDirection));
    // 2. Smoothstep Terminator: Creates a soft dawn/dusk gradient across twilight
    float dayMix = smoothstep(-0.12, 0.15, sunDot);
    // 3. Sunset Rim: Warm golden amber glow along the terminator boundary
    float twilight = (1.0 - smoothstep(0.0, 0.22, abs(sunDot))) * 0.55;
    vec3 sunsetGlow = vec3(1.0, 0.44, 0.16) * twilight;
    // 4. Night Hemisphere: City lights amplified by 2.2x + subtle dark ambient
    vec3 nightLights = nightColor * 2.2;
    vec3 nightAmbient = dayColor * 0.04 + vec3(0.015, 0.02, 0.04);
    vec3 nightSide = nightLights + nightAmbient;
    // 5. Day Hemisphere: Blue marble texture with subtle solar diffuse shading
    vec3 daySide = dayColor * (0.85 + 0.35 * max(sunDot, 0.0));
    // 6. Atmospheric Rim: Rayleigh scattering Fresnel glow on the planetary limb
    float fresnel = pow(1.0 - max(dot(vNormal, vec3(0.0, 0.0, 1.0)), 0.0), 3.5);
    vec3 atmosphereGlow = vec3(0.2, 0.55, 1.0) * fresnel * 0.65 * max(sunDot + 0.2, 0.05);
    gl_FragColor = vec4(mix(nightSide, daySide, dayMix) + sunsetGlow + atmosphereGlow, 1.0);
}
```

**Shader Architectural Highlights:**

- **The Terminator Transition:** smoothstep(-0.12, 0.15, sunDot) avoids a harsh binary line between day and night, smoothly interpolating illumination across dawn and dusk.
- **Golden Sunset Glow:** The sunsetGlow formula activates when abs(sunDot) is near zero (the twilight zone), projecting a warm orange-red band (RGB: 1.0, 0.44, 0.16) mimicking atmospheric light refraction.
- **Night City Lights Amplification:** Multiplying the night texture by 2.2 makes urban centers (New York, London, Tokyo) sparkle vibrantly from orbit.
- **Fresnel Atmospheric Scattering:** pow(1.0 - dot(vNormal, viewDir), 3.5) produces an electric blue halo around the curved horizon, simulating Earth's thin upper atmosphere.

# 3. The Masterpiece: 3D Real-Time Earth Globe Engine (Part 3)

## Step 4: Dual-Sphere Architecture & Procedural Starfield

- **Base Earth Sphere:** A THREE.SphereGeometry(6.8, 64, 64) running the custom GLSL shader material.
- **Atmospheric Cloud Layer:** A secondary, slightly larger concentric sphere of radius **6.87** (0.07 units above the surface). It uses a semi-transparent cloud texture (opacity: 0.35) and THREE.AdditiveBlending. It rotates at an independent rate (clouds.rotation.y += 0.0003) to simulate atmospheric winds.
- **Procedural 3D Starfield:** Generates **2,800 stars** in a THREE.BufferGeometry across an 850-unit cube, rendered via THREE.Points with a subtle orbital drift of 0.00015 rad/frame.

## Step 5: Geographic Coordinate Projection Math

```
function latLongToVector3(lat, lon, radius) {
    const phi = (90 - lat) * (Math.PI / 180);    // Polar angle (0 to PI)
    const theta = (lon + 180) * (Math.PI / 180); // Azimuthal angle (0 to 2*PI)
    return new THREE.Vector3(
        -(radius * Math.sin(phi) * Math.cos(theta)), // X
        radius * Math.cos(phi),                      // Y (Vertical)
        radius * Math.sin(phi) * Math.sin(theta)    // Z
    );
}
```

## Step 6: Interactive Beacons & Canvas Billboards

- **Surface-Normal Cylinder Stem:** A cylinder of height 1.7 units rotated to align with the surface normal vector using quaternion.setFromUnitVectors(new THREE.Vector3(0, 1, 0), normal).
- **Glowing Tip Beacon & Pulse Halo:** A spherical tip mesh (radius 0.36) with outer pulsing halo (radius 0.52).
- **HTML5 2D Canvas Billboards:** Labels are drawn dynamically on an in-memory <canvas> with rounded glass styling and Poppins typography, uploaded to a THREE.CanvasTexture, and rendered as a THREE.Sprite with depthTest: false.

## Step 7: Google Earth "Hyper-Dive" Flight Engine

- **Great-Circle Spherical Interpolation (SLERP):** Camera direction vectors are interpolated using Spherical Linear Interpolation: slerpVectors(startDir, endDir, easedP), preventing the camera from cutting through the Earth.
- **3-Stage Altitude Easing Curve:** High orbital alignment (0-35%) --> Atmospheric plunge (35-85%) --> Touchdown flare (85-100% leveling at surface altitude 1.35).
- **Dynamic FOV Speed "Warp":** Camera FOV dynamically widens from 45 deg up to **53 deg** during the plunge for an exhilarating sense of speed, contracting back to 44 deg on touchdown.
- **Atmospheric Entry Flight HUD:** Glowing cyan HUD overlay with moving laser scanner announces: DIVING INTO MUSA ASSISTANT....
- **Smart Drag-vs-Click Raycasting:** Only pointer movements smaller than **6 pixels** trigger a dive, preventing accidental clicks while rotating the Earth.

# 4. Projects Highlighted Across the Globe

The globe features three core software engineering projects deployed at distinct geographic coordinates, reflecting global development perspectives:

| Project Name | Location & Coordinates | Technical Architecture & Stack |
|---|---|---|
| **MUSA Assistant**<br>**AI & Academic Systems** | Harare, Zimbabwe<br>(Africa)<br>Lat: -19.01, Lon: 29.15<br>Timezone: UTC+2 | Voice- and text-activated AI academic assistant. Integrates with university student portals for automated timetable notifications, exam schedules, and conversational AI queries. Built with Python, NLP pipelines, and REST APIs. |
| **Inventory Management GUI**<br>**Enterprise Desktop OOP** | New York, USA<br>(North America)<br>Lat: 40.71, Lon: -74.00<br>Timezone: UTC-4 | Desktop inventory management platform. Built on strict Object-Oriented Programming (OOP) principles in Python and Tkinter, backed by SQLite atomic transactions, instant search caching, and automated inventory report generators. |
| **Dynamic Data Visualizer**<br>**Web Analytics & Algorithms** | London, United Kingdom<br>(Europe)<br>Lat: 51.50, Lon: -0.12<br>Timezone: UTC+1 | High-throughput browser analytics engine. Features custom Canvas API real-time charting, algorithmic sorting step-by-step visualizations, dataset CSV ingestion, and multi-format statistical reporting. |

## Live Daylight Status Calculator

When the user selects any project on the globe, the application calls getLocalDaylightStatus(timezoneOffset). It queries the client's live UTC clock, applies the target project's timezone offset, and checks whether local time is between 06:00 and 18:00. The project inspector card displays a real-time badge: **[Daylight (14:30 Local)]** or **[Night Lights Active (22:15 Local)]**, reinforcing the dynamic connection between the 3D globe's solar shader and real-world time.

```
function getLocalDaylightStatus(timezoneOffset) {
    const now = new Date();
    const utcHours = now.getUTCHours() + now.getUTCMinutes() / 60;
    let localHour = (utcHours + timezoneOffset) % 24;
    if (localHour < 0) localHour += 24;
    const isDay = localHour >= 6 && localHour < 18;
    const timeStr = `${String(Math.floor(localHour)).padStart(2, "0")}:${String(now.getUTCMinutes()).padStart(2, "0")
}`;
    return {
        isDay,
        timeStr,
        label: isDay ? `Daylight (${timeStr} Local)` : `Night Lights Active (${timeStr} Local)`
    };
}
```

# 5. Presentation Delivery Strategy & Defense Guide

## 5.1 High-Impact 2-Minute Opening Elevator Pitch

*Use this word-for-word script during your initial introduction to hook the audience immediately:*

**Presenter Talking Script: Opening Pitch (Memorize or Read Confidently)**

*"Good morning, esteemed judges, professors, and colleagues. My name is Leroy Matienga, and I am an undergraduate Computer Science student specializing in backend systems engineering and interactive web technologies. Today, I am proud to present my personal portfolio web application. Most developer portfolios are static digital resumes. I wanted mine to be living proof of my engineering capabilities. My portfolio is a full-stack platform built with Python Flask, SQLite, and Three.js WebGL. Its centerpiece is an interactive, real-time 3D Earth globe that runs a custom GLSL fragment shader calculating the exact position of the Sun relative to Earth using live UTC astronomical formulas. Visitors can inspect my software deployments across Africa, North America, and Europe through a cinematic orbital flight engine that models great-circle spherical curves. Behind this visual layer lies an enterprise-grade backend with directory traversal defenses, IP-based sliding-window rate limiting, parameterized SQL queries, and an automated SMTP notification engine. Over the next few minutes, I will walk you through the engineering architecture, the code structure, and the mathematical implementation of the 3D globe."*

## 5.2 Ten-Minute Presentation Roadmap

- **Minute 0:00 - 1:30 | Introduction & Hero Section:** State your name, degree, and career focus. Point out the magnetic pill capsule navigation and typewriter loop. Mention that the CV download streams directly from a protected Flask endpoint.
- **Minute 1:30 - 3:30 | The 3D Earth Globe Showcase (The "Wow" Factor):** Scroll down to the 3D globe. Click Fullscreen. Explain that the day/night boundary is generated live on the GPU using custom GLSL shaders and UTC solar declination formulas. Point out the golden twilight rim and sparkling city lights.
- **Minute 3:30 - 5:30 | Project Deep-Dive via Hyper-Dive:** Click the MUSA Assistant dock pill. Let the audience watch the camera plunge into Harare, Zimbabwe with the animated flight HUD and speed FOV warp. Explain MUSA's AI architecture. Then pull out to orbit and dive into New York or London.
- **Minute 5:30 - 7:30 | Backend Architecture & Database:** Transition to the server architecture. Open or discuss app.py and models.py. Highlight how you mitigated XSS via html.escape(), stopped SQL injection via parameterized queries, and implemented an IP sliding-window rate limiter.
- **Minute 7:30 - 9:00 | Live Contact Form Demonstration:** Fill out the contact form with a test inquiry. Click Send. Show the loading spinner transitioning to the green checkmark toast. Explain that this atomically logged to SQLite and dispatched an encrypted HTML email via smtplib.
- **Minute 9:00 - 10:00 | Conclusion & Q&A:** Summarize your engineering philosophy: clean code, defensive architecture, and immersive user experiences. Open the floor for questions.

# 5.3 Anticipated Technical Defense Questions & Model Answers

*Examiners and professors will test your technical depth. Here are the most likely challenging questions and how to answer them:*

### Q1: Why did you build a custom GLSL shader instead of using standard Three.js MeshStandardMaterial with a directional light?

**Model Answer:** "Standard Three.js materials calculate lighting on a single texture. They cannot simultaneously blend between a daytime satellite map and a nighttime illuminated city lights map while rendering a warm amber twilight glow along the terminator rim. By writing custom GLSL vertex and fragment shaders, I gained direct control over GPU hardware registers. I calculate the dot product between the surface normal and the live solar direction vector, allowing me to smoothly interpolate between night lights and day marble using smoothstep, while computing an atmospheric Rayleigh Fresnel rim glow on the planetary limb."

### Q2: How does the camera flight engine avoid cutting through the Earth during hyper-dives?

**Model Answer:** "If you linearly interpolate camera position from orbit (distance = 24) to the surface (distance = 8.15), the straight chord cuts straight through the Earth's interior geometry. To prevent this, I implemented Spherical Linear Interpolation (SLERP) using unit direction vectors. The direction vector follows a spherical great-circle arc over the globe, while the distance scalar follows an altitude easing curve. The descent is delayed until the camera has rotated horizontally over the target, after which it executes a steep atmospheric plunge and levels off at a flare altitude of 1.35 units above the surface."

### Q3: How do you handle mobile performance and prevent GPU throttling on lower-end devices?

**Model Answer:** "I implemented three key optimizations: First, renderer pixel ratio is clamped using Math.min(window.devicePixelRatio, 2), preventing ultra-high-resolution mobile screens from pushing 4K framebuffers unnecessarily. Second, animations use requestAnimationFrame rather than timers, synchronizing with the display's native refresh rate. Third, all 2,800 stars share a single BufferGeometry, which submits the entire starfield to the GPU in a single draw call."

### Q4: Why choose Flask and SQLite rather than a heavier stack like Django or Node.js/PostgreSQL?

**Model Answer:** "Software engineering is about selecting the right tool for the problem. For a high-performance portfolio, a heavy ORM and complex multi-process relational server adds unnecessary operational overhead and memory footprint. Flask provided a lean, microsecond-fast REST layer with zero bloat. SQLite operates in-process with zero network latency, supports ACID atomic transactions, and handles thousands of daily submissions effortlessly without requiring a dedicated database daemon."

### Q5: How does your rate limiter work without an external database like Redis?

**Model Answer:** "In models.py, I engineered a sliding-window rate query using SQLite's built-in datetime engine: WHERE ip_address = ? AND created_at >= datetime('now', '-60 seconds'). On every contact submission, app.py queries this count. If the IP has made 5 or more requests within the last 60 seconds, it immediately rejects the request with HTTP 429 Too Many Requests. This gives me sub-millisecond rate limiting with zero external dependencies."

### Q6: How did you ensure the contact form wouldn't fail if the email service goes down?

**Model Answer:** "I applied the principle of Graceful Degradation. The database insert and the email notification are decoupled. When a user submits a message, the database write occurs first. Then, the email dispatch in email_service.py is wrapped in a non-blocking try-except block. If SMTP credentials expire or the network times out, the server logs an error to stderr but returns HTTP 201 to the client. The user's message is safely stored in SQLite, and the system never crashes."

# 5.4 Final Defense Preparation & Live Demo Checklist

To ensure a flawless presentation tomorrow, follow this step-by-step technical rehearsal procedure:

- **Step 1: Start Server in Local Terminal:** Open terminal in the portfolio directory and run: python app.py. Confirm the banner output: Leroy Matienga Portfolio Server Running on http://localhost:5000.
- **Step 2: Browser Setup & Viewport:** Open Google Chrome or Microsoft Edge and navigate to http://localhost:5000. Press **F11** to enter clean borderless fullscreen.
- **Step 3: Test Navigation Indicator:** Move your cursor across Home, About, Portfolio, Skills, and Contact. Click and drag across the navbar to show the magnetic indicator tracking.
- **Step 4: Demonstrate 3D Globe Flight:** Scroll to the 3D globe. Click the **Fullscreen** button on the top-right of the space frame. Point out the live Sun indicator. Click **MUSA Assistant (Africa)** to trigger the hyper-dive flight with the HUD scanner.
- **Step 5: Inspect Project Details:** Show the daylight badge (e.g., ■■ Daylight or ■ Night Lights) and explain how the UTC clock informs local time.
- **Step 6: Return to Orbit:** Click the **Orbit View** button or press the **Escape** key to trigger the smooth pull-out camera transition.
- **Step 7: Demonstrate Defensive Backend Form:** Scroll to the Contact section. Enter a test name, email, subject, and message. Submit and observe the loading state and green success feedback toast.
- **Step 8: Demonstrate Rate Limiter (Optional Power Move):** If an examiner asks about security, rapidly click submit multiple times or trigger 6 API calls in Postman/curl to show the server cleanly returning HTTP 429 Too Many Requests.

**[SUMMARY & FINAL PRESENTATION ADVICE]**

Leroy, you built a truly impressive, multi-disciplinary software project. It balances advanced 3D computer graphics, astronomical physics, custom GPU shaders, robust security patterns, and clean Python backend engineering. Speak with confidence, take your time during the 3D globe flight, and remember that you understand every single line of this codebase. Best of luck with tomorrow's presentation!

| | | |
|---|---|---|
| **Presenter:** Leroy Matienga<br>Candidate for B.Sc. Computer Science | **Project:** Interactive 3D Portfolio Application<br>Full-Stack Architecture & WebGL Engine | **Status: Presentation Ready**<br>**Document Generated: October 2026** |